



# **Rapport – TP3 : Gestion de la latence**

Groupe 12

Christian Akkary - 2152001

Tomiwa Orolu - 2178786

Yann Pernot - 2486801

Avril 2026

# I. Implémentation

## Prédiction des cercles

*CircleGhost* prédit la position des cercles en simulant (*localTick* - *serverTick*) ticks en avance depuis la dernière position connue du serveur. L'offset par tick est préféré au RTT car il est déterministe et directement aligné sur les données reçues, indépendamment de l'horloge réseau. La simulation (*SimulateCircleTick*) reproduit exactement la physique de *MovingCircle* : déplacement par vitesse et réflexion aux bords avec *Time.fixedDeltaTime*.

## Prédiction d'inputs et réconciliation

Chaque input WASD est appliqué immédiatement à *m\_PredictedPosition* et conservé dans *m\_InputHistory*. À chaque confirmation de tick serveur (*m\_ServerConfirmedTick*), l'historique est purgé jusqu'au tick confirmé et les inputs restants sont rejoués sur la position autoritaire du serveur. Une simple téléportation vers la position serveur aurait produit des corrections visibles à chaque RTT en l'absence d'input adverse.

## Prédiction du stun

*GameState.IsStunned* retourne *m\_IsStunned.Value* || *m\_IsLocallyStunned*. Quand le client appuie sur espace, *m\_IsLocallyStunned* est activé immédiatement, puis effacé à réception de la confirmation serveur. Tous les systèmes existants honorent ainsi la prédiction sans modification.

## Fluidité du stun

Au début du stun, *CircleGhost* capture *m\_FrozenPosition* via *transform.localPosition* plutôt que *m\_MovingCircle.Position*, évitant un recul visuel. En fin de stun, le retard du *m\_ServerTick* causerait une téléportation vers l'avant. Pour éviter ce problème, *CircleGhost* mémorise *m\_StunEndLocalTick* et calcule *ticksToPredict* = *localTick* - *m\_StunEndLocalTick*, clampé à *preStunTicks*. La prédiction repart donc de zéro et croît progressivement, ce qui permet de lisser la transition.

## II. Maintenabilité

La duplication de logique de simulation entre *MovingCircle/CircleGhost* et les chemins serveur/client de *Player* constitue le principal risque : toute modification physique doit être répercutée manuellement des deux côtés. Centraliser ces fonctions en méthodes statiques partagées réduirait ce couplage implicite, au coût d'une légère complexification structurelle.